

DATABASE SYSTEMS

Chap 6~Parallel and Distributed Databases



0. INTRODUCTION

- ❖ We have thus far **considered centralized database management systems** in which:
 - ✓ all the details maintained at a **single site** and
 - ✓ assumed that the processing of individual transactions is **essentially sequential**.
- ❖ One of the most important **trends** in databases is the increased use of parallel evaluation techniques and data distribution.



1. PARALLEL DATABASES

- ❖ Parallel databases is all about machines which are **physically close to each other** (e.g in the same server room) connected with dedicated high-speed LANs and switches where communication cost is assumed to be so small and at times can be ignored.
- ❖ The database can do multiple tasks in parallel allowing the database **to make use of multiple CPU cores and multiple disks that are standard for modern database servers**. However, all CPU cores can directly address all disks in a parallel database.



1. PARALLEL DATABASES

- ❖ Maintain the same execution time but increase the overall performance(throughput).

Example:

1 processor = 100 seconds to execute a query

4 processors = 25 seconds to execute a query

$$\text{speed up} = \text{Speedup} = \frac{\text{Time on 1 processor}}{\text{Time on n processors}} = 4$$



1. Architecture of Parallel Databases

1. Shared Memory Architecture

Processor 1

Processor 2 ————— Shared Memory & Shared Disks

Processor 3

2. Shared Disk Architecture

CPU1 + Memory1

CPU2 + Memory2 ————— Shared Disks

CPU3 + Memory3

3. Shared Nothing Architecture(Each processor owns Its own CPU , Its own memory , Its own disk) communicate through network.



Types of Parallelism

❖ Types of Parallelism

1. **Inter-Query Parallelism:** Different queries execute simultaneously (**SELECT * FROM Customer**, **SELECT * FROM Product**).

P1: SELECT * FROM Customer

P2: SELECT * FROM Product

2. **Intra-Query Parallelism:** One query is divided into smaller tasks and multiple processors work on it together (**SELECT SUM(Salary) FROM Employee**). Each processor select some specifics rows.



2. DISTRIBUTED DATABASES

- ❖ Distributed databases comes as a result of databases stored on machines **which are far from each other** (say are located in different continent) connected using public network (like internet) the communication costs cannot be ignored.
- ❖ These data are distributed on different hosts where the CPU will only allow disks on which it belongs.
- ❖ The main reason for this move:
 - Increased globalization of business operations.
 - Accumulation of massive organizational data
 - Technological Evolutions



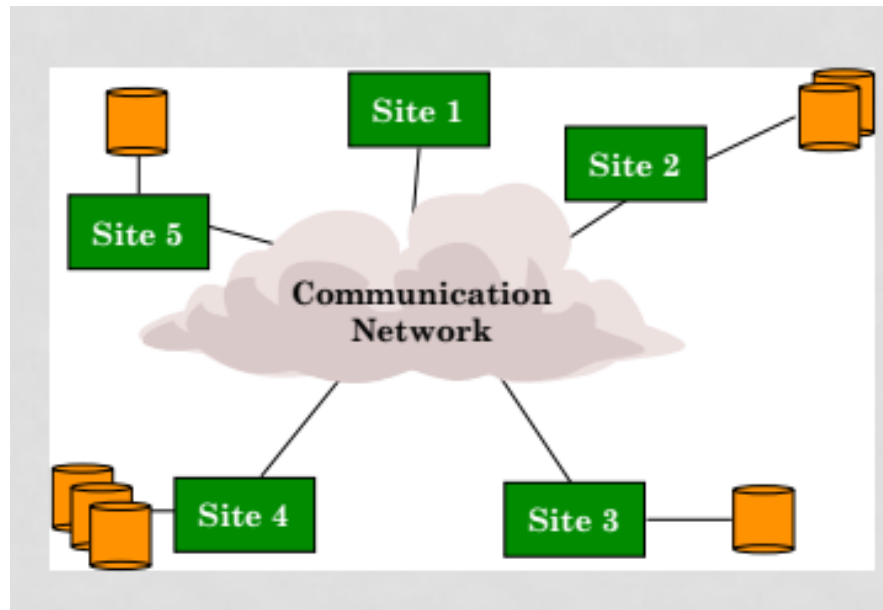
CONT...

- ❖ With distributed databases , databases are hosted on diversified locations each independent but put in place a mechanism for the users to have a feeling as if they are working a single database.



1.1 Structure of Distributed Database

- A distributed database (DDB) is a collection of multiple, **logically interrelated** databases distributed over a **computer network**.



- A distributed Database Management System (D-DBMS) is the software that manages the DDB and provides an access mechanism that makes this distribution **transparent** to the users.
- Distributed Database System (DDBS) = DB + Communication*



1.2 Distributed Databases Main Concepts

- Data are stored at several locations
 - *Each managed by a DBMS that can run autonomously*
- Ideally, location of data is unknown to client
 - *Distributed data Independence*
- Distributed Transactions.
 - *Clients can write transactions regardless of where the affected data are located*



1.3 Distributed DBMS Abilities

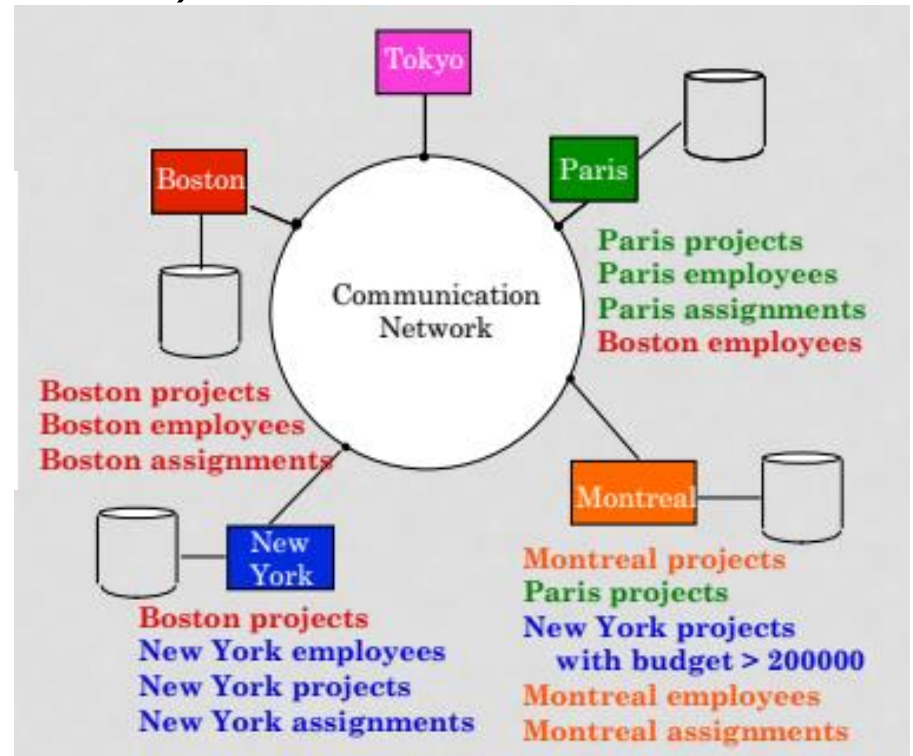
- Transparent management of distributed, fragmented, and replicated data.
- Improved reliability/availability through distributed transactions
- Improved performance
- Easier and more economical system expansion



1.4 Transparency & Data Independence

- Data distributed (with some replication)
- Transparently ask query:

```
SELECT ENAME, SAL
FROM   EMP, ASG, PAY
WHERE  DUR > 12
AND    EMP.ENO = ASG.ENO
AND    PAY.TITLE = EMP.TITLE
```

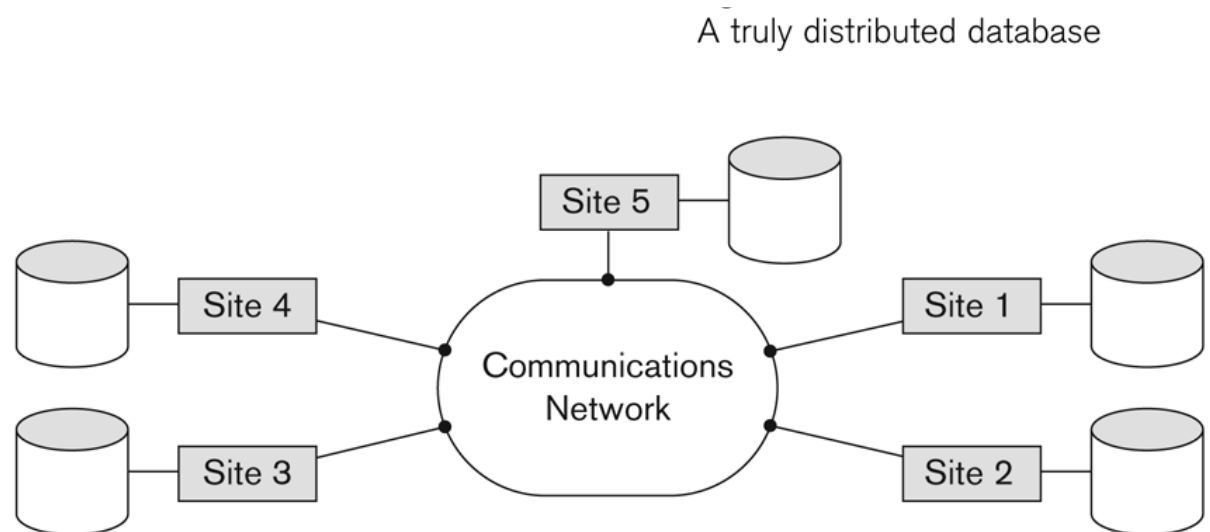




1.5 Advantages

(a) Management of distributed data with different **levels of transparency:**

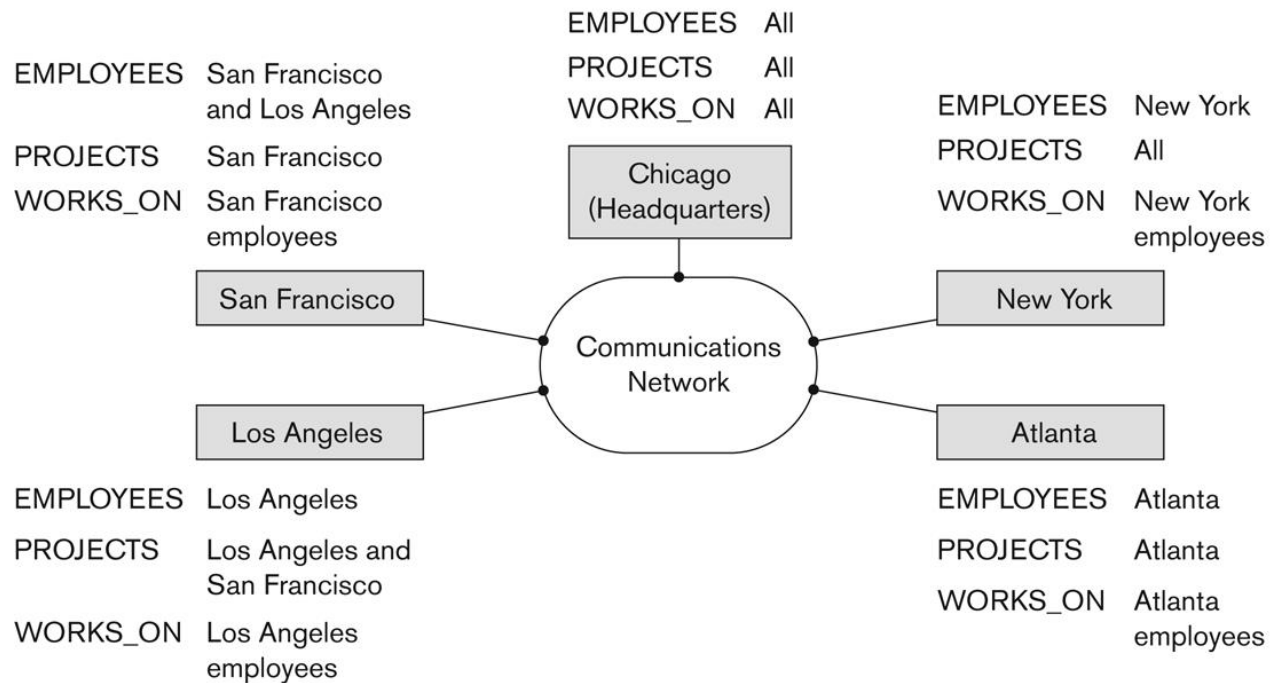
- This refers to the physical placement of data (files, relations, etc.) which is not known to the user (distribution transparency).





Advantages (cont'd...)

- E.g., The EMPLOYEE, PROJECT, and WORKS_ON tables may be fragmented horizontally and stored with possible replication as shown below.





Advantages (cont'd...)

(b) Distribution and Network transparency:

- Users do not have to worry about operational details of the network.
 - There is Location transparency, which refers to freedom of issuing command from any location without affecting its working.
 - Then there is Naming transparency, which allows access to any names object (files, relations, etc.) from any location.



Advantages (cont'd...)

(c) **Replication transparency:**

- It allows to store copies of a data at multiple sites as shown in the above diagram.
- This is done to minimize access time to the required data.

(d) **Fragmentation transparency:**

- Allows to fragment a relation horizontally (create a subset of tuples of a relation) or vertically (create a subset of columns of a relation).



Advantages (cont'd...)

(e) **Increased reliability and availability:**

- **Availability** is the probability that the system is continuously available (usable or accessible) during a time interval.
- A distributed database system has multiple nodes (computers) and if one fails then others are available to do the job.



Advantages (cont'd...)

(f) Improved performance:

- A distributed DBMS fragments the database to keep data closer to where it is needed most.
- This reduces data management (access and modification) time significantly.

(g) Easier expansion (scalability):

- Allows new nodes (computers) to be added anytime without changing the entire configuration.



2. Data Fragmentation, Replication and Allocation

- **How to divide the data? Can't we just distribute relations?**
- What is a reasonable unit of distribution?
 - Relation
 - Views are subsets of relations
 - Extra communication
 - Less parallelism
 - Fragments of relations (sub-relations)



2.1 Data Fragmentation, Replication and Allocation

- Split a relation into logically related and correct parts.
- A relation can be fragmented in two ways:
 - **Horizontal Fragmentation**
 - **Vertical Fragmentation**
 - Given a relation

PNO	PNAME	BUDGET	LOC
P1	Instrumentation	150000	Montreal
P2	Database Develop.	135000	New York
P3	CAD/CAM	250000	New York
P4	Maintenance	310000	Paris
P5	CAD/CAM	500000	Boston



2.1 Data Fragmentation, Replication and Allocation

- **Horizontal fragmentation**

- It is a horizontal subset of a relation which contain those tuples which satisfy [selection conditions](#).
- A selection condition may be composed of several conditions connected by AND or OR.
- Derived horizontal fragmentation: It is the partitioning of a primary relation to other secondary relations which are related with Foreign keys.



2.1 Data Fragmentation, Replication and Allocation

- Horizontal fragmentation

$PROJ_1$: projects with budgets less than \$200,000

$PROJ_2$: projects with budgets greater than or equal to \$200,000

PNO	PNAME	BUDGET	LOC
P1	Instrumentation	150000	Montreal
P2	Database Develop.	135000	New York
P3	CAD/CAM	250000	New York
P4	Maintenance	310000	Paris
P5	CAD/CAM	500000	Boston

$PROJ_1$

PNO	PNAME	BUDGET	LOC
P1	Instrumentation	150000	Montreal
P2	Database Develop.	135000	New York

Stored in London

$PROJ_2$

PNO	PNAME	BUDGET	LOC
P3	CAD/CAM	250000	New York
P4	Maintenance	310000	Paris
P5	CAD/CAM	500000	Boston

Stored in Boston



2.1 Data Fragmentation, Replication and Allocation

- **Vertical fragmentation**

- It is a subset of a relation which is created by a subset of columns.
- Thus a vertical fragment of a relation will contain values of selected columns.
- There is no selection condition used in vertical fragmentation.
- Because there is no condition for creating a vertical fragment, each fragment must include the primary key attribute of the parent relation Employee.
- In this way all vertical fragments of a relation are connected.



2.1 Data Fragmentation, Replication and Allocation

- Vertical fragmentation

$PROJ_1$: information about project budgets

$PROJ_2$: information about project names and locations

PROJ

PNO	PNAME	BUDGET	LOC
P1	Instrumentation	150000	Montreal
P2	Database Develop.	135000	New York
P3	CAD/CAM	250000	New York
P4	Maintenance	310000	Paris
P5	CAD/CAM	500000	Boston

$PROJ_1$

PNO	BUDGET
P1	150000
P2	135000
P3	250000
P4	310000
P5	500000

Stored in London

$PROJ_2$

PNO	PNAME	LOC
P1	Instrumentation	Montreal
P2	Database Develop.	New York
P3	CAD/CAM	New York
P4	Maintenance	Paris
P5	CAD/CAM	Boston

Stored in Boston

15



2.1 Data Fragmentation, Replication and Allocation

- **Representation**
 - **Horizontal fragmentation**
 - Each horizontal fragment on a relation can be specified by a $\sigma_{C_i}(R)$ operation in the relational algebra.
 - Complete horizontal fragmentation
 - A set of horizontal fragments whose conditions $C_1, C_2, \dots, C_n$ include all the tuples in R - that is, every tuple in R satisfies (C_1 OR C_2 OR ... OR C_n).
 - Disjoint complete horizontal fragmentation: No tuple in R satisfies (C_i AND C_j) where $i \neq j$.
 - To reconstruct R from horizontal fragments a UNION is applied.



2.1 Data Fragmentation, Replication and Allocation

- **Representation**

- **Vertical fragmentation**

- A vertical fragment on a relation can be specified by a $\Pi_{L_i}(R)$ operation in the relational algebra.
 - Complete vertical fragmentation
 - A set of vertical fragments whose projection lists $L_1, L_2, \dots, L_n$ include all the attributes in R but share only the primary key of R . In this case the projection lists satisfy the following two conditions:
 - $L_1 \cup L_2 \cup \dots \cup L_n = \text{ATTRS}(R)$
 - $L_i \cap L_j = \text{PK}(R)$ for any i, j , where $\text{ATTRS}(R)$ is the set of attributes of R and $\text{PK}(R)$ is the primary key of R .
 - To reconstruct R from complete vertical fragments a OUTER UNION is applied.



2.1 Data Fragmentation, Replication and Allocation

- **Representation**

- **Mixed (Hybrid) fragmentation**

- A combination of Vertical fragmentation and Horizontal fragmentation.
 - This is achieved by **SELECT-PROJECT operations** which is represented by $\Pi_{L_i}(\sigma_{C_i}(R))$.
 - If $C = \text{True}$ (Select all tuples) and $L \neq \text{ATTRS}(R)$, we get a vertical fragment, and if $C \neq \text{True}$ and $L \neq \text{ATTRS}(R)$, we get a mixed fragment.
 - If $C = \text{True}$ and $L = \text{ATTRS}(R)$, then R can be considered a fragment.



2.1 Data Fragmentation, Replication and Allocation

- **Fragmentation schema**

- A definition of a set of fragments (horizontal or vertical or horizontal and vertical) that includes all attributes and tuples in the database that satisfies the condition that the whole database can be reconstructed from the fragments by applying some sequence of UNION (or OUTER JOIN) and UNION operations.

- **Allocation schema**

- It describes the distribution of fragments to sites of distributed databases.
- It can be fully or partially replicated or can be partitioned.
- Fragmentation and replication **(fast access and fault tolerance)**



2.2.Data Fragmentation, Replication and Allocation

- **Data Replication**
 - Database is replicated to all sites.
 - In **full replication** the entire database is replicated and in **partial replication** some selected part is replicated to some of the sites.
 - Data replication is achieved through a replication schema.



2.3 Data Fragmentation, Replication and Allocation

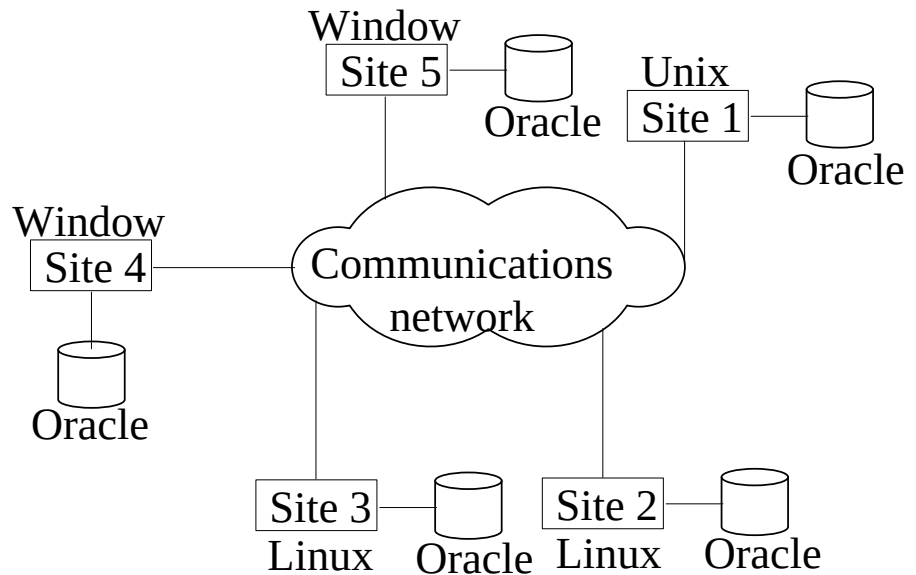
- **Data Distribution (Data Allocation)**
 - This is relevant only in the case of partial replication or partition.
 - The selected portion of the database is distributed to the database sites.
 - Between below allocation which one is best in term of storage, data availability, and data security and backup:
 1. Centralized Allocation
 2. Replicated Allocation
 3. Partitioned Allocation



3. Types of Distributed Database Systems

- **Homogeneous**

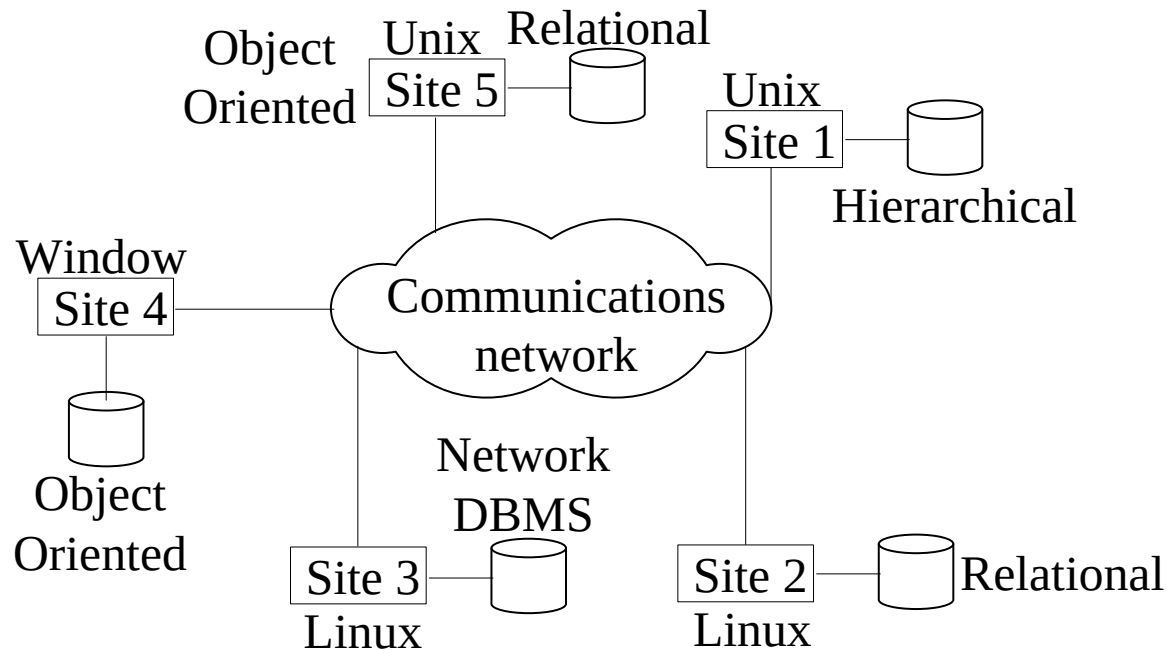
- All sites of the database system have identical setup, database model i.e., same database system software.
- The underlying operating system may be different.
 - For example, all sites run Oracle or DB2, or Sybase or some other database system.
- The underlying operating systems can be a mixture of Linux, Window, Unix, etc.





Types of Distributed Database Systems

- **Heterogeneous**
 - Different sites run different DBMS (maybe even RDBMS and ODBMS)





3. Types of Distributed Database Systems

- **Heterogeneous**

- **Federated**: Each site may run different database system but the data access is managed through a single conceptual schema.

- This implies that the degree of local autonomy is minimum.

- Each site must adhere to a centralized access policy.

- sites still control their DBs

- Site A → Oracle (HR system)

- Site B → MySQL (Sales system)

- Site C → PostgreSQL (Inventory system)

- **Multidatabase**: There is no one conceptual global schema.

- For data access, a schema is constructed dynamically as needed by the application software.

- Bank DB → Oracle

- Insurance DB → MySQL

- Tax Authority DB → PostgreSQL



Types of Distributed Database Systems

- Federated Database Management Systems Issues
 - Differences in data models:
 - Relational, Objected oriented, hierarchical, network, etc.
 - Differences in constraints:
 - Each site may have their own data accessing and processing constraints.
 - Differences in query language:
 - Some site may use SQL, some may use SQL-89, some may use SQL-92, and so on.



4. Query Processing in Distributed Databases

- Issues

- Cost of transferring data (files and results) over the network.

- This cost is usually high so some optimization is necessary.
- Example relations: Employee at site 1 and Department at Site 2
 - Employee at site 1. 10,000 rows. Row size = 100 bytes. Table size = 10^6 bytes.

Fname	Minit	Lname	<u>SSN</u>	Bdate	Address	Sex	Salary	Superssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	----------	-----

- Department at Site 2. 100 rows. Row size = 35 bytes. Table size = 3,500 bytes.

Dname	Dnumber	Mgrssn	Mgrstartdate
-------	---------	--------	--------------

- Q: For each employee, retrieve employee name and department name Where the employee works.
- Q: $\Pi_{\text{Fname, Lname, Dname}} (\text{Employee} \bowtie_{\text{Dno} = \text{Dnumber}} \text{Department})$



Query Processing in Distributed Databases

- Result
 - The result of this query will have 10,000 tuples, assuming that every employee is related to a department.
 - Suppose each result tuple is 40 bytes long. The query is submitted at site 3 and the result is sent to this site.
 - **Problem:** Employee and Department relations are not present at site 3.



Query Processing in Distributed Databases

- Strategies:
 1. Transfer Employee and Department to site 3.
 - Total transfer bytes = $1,000,000 + 3500 = 1,003,500$ bytes.
 2. Transfer Employee to site 2, execute join at site 2 and send the result to site 3.
 - Query result size = $40 * 10,000 = 400,000$ bytes. Total transfer size = $400,000 + 1,000,000 = 1,400,000$ bytes.
 3. Transfer Department relation to site 1, execute the join at site 1, and send the result to site 3.
 - Total bytes transferred = $400,000 + 3500 = 403,500$ bytes.
- Optimization criteria: minimizing data transfer.



Query Processing in Distributed Databases

- Strategies:
 1. Transfer Employee and Department to site 3.
 - Total transfer bytes = $1,000,000 + 3500 = \mathbf{1,003,500 \text{ bytes}}$.
 2. Transfer Employee to site 2, execute join at site 2 and send the result to site 3.
 - Query result size = $40 * 10,000 = 400,000$ bytes. Total transfer size = $400,000 + 1,000,000 = \mathbf{1,400,000 \text{ bytes}}$.
 3. Transfer Department relation to site 1, execute the join at site 1, and send the result to site 3.
 - Total bytes transferred = $400,000 + 3500 = \mathbf{403,500 \text{ bytes}}$.
 - Optimization criteria: minimizing data transfer.
 - Preferred approach: strategy 3.
-



Query Processing in Distributed Databases

- Now suppose the result site is 2. Possible strategies :
 1. Transfer Employee relation to site 2, execute the query and present the result to the user at site 2.
 - Total transfer size = 1,000,000 bytes for both queries Q and Q'.
 2. Transfer Department relation to site 1, execute join at site 1 and send the result back to site 2.
 - Total transfer size for Q = $400,000 + 3500 = 403,500$ bytes and for Q' = $4000 + 3500 = 7500$ bytes.
-

QUESTION

Assume a baking system having relations: Customer at site 1 and Account at Site 2
Customer at site 1. 220 rows. Row size = 25 bytes and Account at site 2. 9000 rows. Row size = 80 bytes.

As a query, we need to retrieve all customer names and their Account number, assume that result of each truple has 60bytes.

- What is the best optimization approach if the result has to be submitted to site 4
Transfer Customer and Account to the result site and perform the join at site 4.
 - Transfer Customer to site 2, execute join at site 2 and send the result to site 4.
 - Transfer Account relation to site 1, execute join at site 1 and send the result to site 4.
- What is the best optimization approach if the result has to be submitted to site 2?
Transfer Customer relation to site 2, execute the query and present the result to the user at site 2.
 - Transfer Account relation to site 1, execute join at site 1 and send the result back to site 2